

Computer Networks

Transport, Application & Emerging Concepts

Ch 5: Transport Layer — TCP, UDP, Port Addressing, Congestion Control

Ch 6: Application Layer — DNS, HTTP, FTP, SMTP, POP3, IMAP, SSH

Ch 7: Emerging Concepts — Cloud, IoT, MQTT, CoAP, SDN

Indranil Ganguly

Subject: Computer Networks | Semester Exam Revision

Easy Language + Real-World Analogies

Prepared with Claude AI

Table of Contents

CHAPTER 5: TRANSPORT LAYER

- 5.1 What is the Transport Layer?
- 5.2 Functions of the Transport Layer
- 5.3 TCP — Transmission Control Protocol
- 5.4 TCP 3-Way Handshake (Connection Setup)
- 5.5 TCP 4-Way Handshake (Connection Teardown)
- 5.6 UDP — User Datagram Protocol
- 5.7 TCP vs UDP Comparison
- 5.8 Port Addressing
- 5.9 Congestion Control — Leaky Bucket
- 5.10 Congestion Control — Token Bucket

CHAPTER 6: APPLICATION LAYER PROTOCOLS

- 6.1 DNS — Domain Name System
- 6.2 HTTP & HTTPS
- 6.3 FTP — File Transfer Protocol
- 6.4 Email Protocols: SMTP, POP3, IMAP
- 6.5 Remote Login: Telnet & SSH
- 6.6 File Sharing: NFS & SMB
- 6.7 SNMP — Simple Network Management Protocol

CHAPTER 7: EMERGING NETWORKING CONCEPTS

- 7.1 Cloud Networking Basics
- 7.2 Internet of Things (IoT)
- 7.3 MQTT Protocol
- 7.4 CoAP Protocol
- 7.5 Software-Defined Networking (SDN)
- Quick Revision Summary

Chapter 5: Transport Layer

5.1 What is the Transport Layer?

The **Transport Layer** is Layer 4 of the OSI model. It sits between the Application Layer (above) and the Network Layer (below). While the Network Layer delivers packets from computer to computer, the Transport Layer delivers data from **application to application** — process to process.

Real-world analogy: Think of the Network Layer as the post office delivering a parcel to a building. The Transport Layer is the building's receptionist who figures out which flat (which application) the parcel belongs to, and hands it to the right person.

The Big Picture

- Network Layer delivers packet: Computer A --> Computer B
- Transport Layer delivers data: Chrome browser --> Web server (process-to-process)
- Two main protocols: TCP (reliable, ordered) and UDP (fast, no guarantees)
- Uses PORT NUMBERS to identify which application should receive the data

5.2 Functions of the Transport Layer

Process-to-Process Delivery

Network Layer gets data to the right computer. Transport Layer then gets it to the right APPLICATION running on that computer using port numbers. Example: Port 80 = web browser, Port 25 = email.

Segmentation & Reassembly

Big data files are too large to send in one go. The Transport Layer breaks them into smaller pieces called SEGMENTS. At the other end, it reassembles them in the correct order. Like tearing a long letter into pages, mailing each page separately, then re-reading them in order.

Connection Management

TCP establishes a connection before sending data (3-way handshake) and closes it properly after (4-way handshake). Like calling someone before sending a long fax: 'Are you ready?' — 'Yes, ready.' — 'Sending now.'

Flow Control

The sender should not send data faster than the receiver can process. Like a teacher speaking slowly so students can take notes — Transport Layer tells the sender to slow down if the receiver's buffer is full.

Error Control

Ensures data arrives correctly. Uses checksums to detect errors. If a segment is lost or corrupted, it requests retransmission. Like asking 'Sorry, could you repeat that?' in a noisy phone call.

Congestion Control

If the network is overloaded (too much traffic), the Transport Layer slows down the sender to reduce congestion. Like cars slowing down near a toll booth to avoid gridlock.

Multiplexing & Demultiplexing

Multiplexing: Multiple applications (Chrome, Skype, WhatsApp) can send data simultaneously using the same network connection — each using different ports. Demultiplexing: At receiver, Transport Layer reads port number and delivers each segment to the correct application.

5.3 TCP — Transmission Control Protocol

TCP is a **connection-oriented, reliable** protocol. It guarantees that every byte of data arrives at the destination, in the correct order, without errors. It is used whenever accuracy is more important than speed.

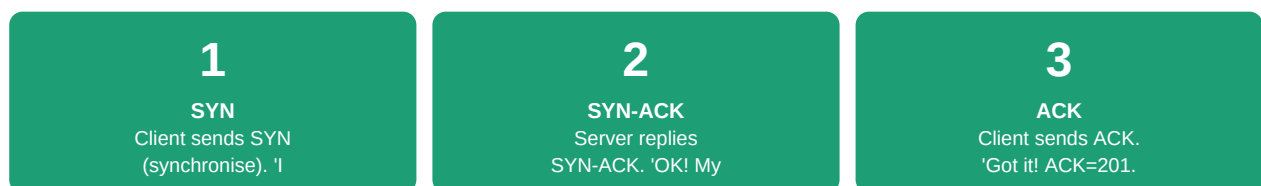
Real-world analogy: TCP is like sending a registered post parcel. You get an acknowledgement when it reaches the destination. If the parcel is lost, the courier re-sends it. Slower than ordinary post, but guaranteed delivery.

- Connection-oriented: Must establish a connection before sending any data
- Reliable: Every segment is acknowledged (ACK). Lost segments are retransmitted
- Ordered delivery: Segments are numbered; receiver reorders if they arrive out-of-order
- Flow control: Uses sliding window to avoid overwhelming the receiver
- Congestion control: Slows down if network is congested (Slow Start, Congestion Avoidance)
- Full duplex: Both sides can send and receive simultaneously
- Used for: Web browsing (HTTP/HTTPS), Email (SMTP), File transfer (FTP), SSH

5.4 TCP 3-Way Handshake — How a Connection is ESTABLISHED

Before TCP sends any data, it must first establish a connection between client and server using a 3-step process called the **3-Way Handshake**.

Real-world analogy: Imagine you call a friend: (1) You: 'Hello, can you hear me?' — (2) Friend: 'Yes I can hear you, can you hear me?' — (3) You: 'Yes!' Now both sides know the line is working. That's the 3-way handshake!



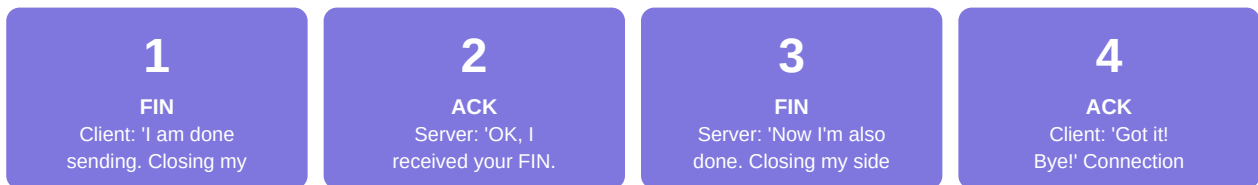
Step	Who Sends	Message	What it Means
1	Client→Server	SYN	I want to connect. My starting sequence number is X
2	Server→Client	SYN + ACK	OK! I'm ready (seq=Y). I got your seq (ack=X+1)
3	Client→Server	ACK	Great! Got your seq (ack=Y+1). Connection open now!

Real-world analogy: After 3-Way Handshake: Both sides agree on sequence numbers. Data transfer can begin. Connection is ESTABLISHED.

5.5 TCP 4-Way Handshake — How a Connection is CLOSED

When data transfer is complete, TCP closes the connection gracefully using a **4-Way Handshake**. Either side can initiate the closing.

Real-world analogy: Like ending a phone call politely: (1) You: 'I'm done, goodbye' — (2) Friend: 'OK, noted.' — (3) Friend: 'I'm done too, goodbye' — (4) You: 'OK bye!' Now the call is fully ended.



Step	Who Sends	Message	Meaning
1	Client→Server	FIN	I have finished sending data. Please close my side.
2	Server→Client	ACK	OK, I got your FIN. My side may still send data.
3	Server→Client	FIN	Now I am also done. Please close my side too.
4	Client→Server	ACK	Got it. Connection fully closed now.

Real-world analogy: Why 4 steps instead of 3? Because the server might still have data to send after receiving the client's FIN — so it ACKs first, finishes sending, then sends its own FIN.

5.6 UDP — User Datagram Protocol

UDP is a **connectionless, unreliable** protocol. It sends data as fast as possible without establishing a connection or waiting for acknowledgements. No guarantees — some packets may be lost, and that's acceptable.

Real-world analogy: UDP is like sending an ordinary postcard. You just write and drop it in the postbox — no tracking, no confirmation of delivery. It might get lost. But it's super fast and cheap. Perfect for a quick message!

- Connectionless: No handshake, no connection setup — just send the data
- Unreliable: No acknowledgements, no retransmission of lost packets
- No ordering: Packets may arrive out of order — no reordering done
- No flow control: Sender sends at full speed regardless of receiver capacity
- Very fast: Minimal overhead — small header (only 8 bytes vs TCP's 20 bytes)
- Used for: Live video/audio streaming, online gaming, DNS lookups, VoIP (Zoom, WhatsApp calls)

5.7 TCP vs UDP — Full Comparison

Feature	TCP	UDP
Full Name	Transmission Control Protocol	User Datagram Protocol
Connection	Connection-oriented (3-way handshake)	Connectionless (no setup)
Reliability	Reliable (ACK + retransmit)	Unreliable (no ACK)
Ordering	Guaranteed order	No ordering
Speed	Slower (overhead)	Faster (no overhead)
Header Size	20 bytes minimum	8 bytes (fixed)
Flow Control	Yes (sliding window)	No
Congestion Control	Yes	No
Error Checking	Yes (checksum + ACK)	Checksum only
Use Case	Web, Email, File Transfer, SSH	Video stream, DNS, Gaming, VoIP
Example Protocol	HTTP, FTP, SMTP, SSH	DNS, RTP, DHCP, SNMP
Analogy	Registered Post (guaranteed)	Postcard (fast, no guarantee)

5.8 Port Addressing

A **port number** is a 16-bit number (0–65535) that identifies a specific application or service on a computer. While an IP address identifies the computer, the port number identifies the specific application on that computer.

Real-world analogy: An IP address is like the address of a large apartment building. The port number is the flat number inside. The postman (router) delivers to the building (IP), but the letter goes to a specific flat (port/application).

Three ranges of port numbers:

Range	Name	Description	Who Uses It
0 – 1023	Well-Known Ports	Reserved for standard Internet services	HTTP=80, HTTPS=443, FTP=21, SSH=22, SMTP=25, DNS=53
1024 – 49151	Registered Ports	Used by companies for their applications	MySQL=3306, PostgreSQL=5432, MongoDB=27017
49152 – 65535	Dynamic/Private	Temporary ports assigned by OS	Created when you open Chrome, WhatsApp etc.

Common Well-Known Ports to Remember

Port	Protocol	Service	Analogy
20, 21	FTP	File Transfer	Cargo ship delivering files
22	SSH	Secure Remote Login	Secure telephone line
23	Telnet	Unsecure Remote Login	Open phone call (not private)
25	SMTP	Sending emails	Post office for sending letters
53	DNS	Domain Name lookup	Phone book / directory
67/68	DHCP	IP address assignment	Hotel receptionist giving room keys
80	HTTP	Web browsing (unsecure)	Open newspaper
110	POP3	Receiving emails (downloads)	Collecting your post from letterbox
143	IMAP	Receiving emails (server-stored)	Reading post at the post office
443	HTTPS	Secure web browsing	Sealed, encrypted letter
3306	MySQL	Database	Filing cabinet in office

5.9 Congestion Control — Leaky Bucket Algorithm

Congestion happens when too much data floods the network at once, causing routers to drop packets. **The Leaky Bucket algorithm** is a traffic shaping method that controls the rate at which data enters the network — smoothing out bursts into a steady, predictable flow.

Real-world analogy: Imagine a bucket with a small hole at the bottom. You can pour water in at ANY rate (bursty traffic) — but the water leaks out at a CONSTANT rate (controlled output). If you pour too much, the bucket overflows (packets dropped). The output is always steady, no matter how irregular the input is.

How Leaky Bucket Works

- Incoming packets (bursty or irregular) are put into a buffer (the 'bucket')
- Packets leave the bucket at a FIXED, CONSTANT rate — like water dripping from a hole
- If the bucket is full and more packets arrive → they are DROPPED (overflow)
- Output is always smooth and predictable, regardless of input pattern
- Rate of output = fixed (e.g., always 10 packets per second)

Parameter	Leaky Bucket
Output rate	Fixed and constant always
Burst handling	Absorbs small bursts up to bucket capacity
Overflow	Packets dropped if bucket is full
Effect	Converts bursty traffic into smooth, uniform traffic
Use case	ATM networks, traffic policing in ISPs
Limitation	Cannot handle genuine bursty traffic efficiently — bursts always lost

5.10 Congestion Control — Token Bucket Algorithm

The **Token Bucket algorithm** is more flexible than Leaky Bucket. It allows controlled bursts of traffic while still limiting the average rate.

Real-world analogy: Imagine a bucket filling up with tokens (coins) at a steady rate. Each time you want to send a packet, you must take a token from the bucket. If there are tokens available, you can send — even multiple packets at once (burst). If the bucket is empty, you must wait. No tokens = no sending.

How Token Bucket Works

- Tokens are added to the bucket at a FIXED rate (e.g., 10 tokens/second)
- Bucket has a maximum capacity — tokens that exceed the limit are DISCARDED
- To send a packet, the sender must TAKE one token from the bucket
- If tokens are available → send immediately (bursts are ALLOWED!)
- If no tokens → wait until tokens arrive (sender is throttled)
- Average rate is controlled, but SHORT BURSTS are permitted up to bucket size

Feature	Leaky Bucket	Token Bucket
Output rate	Strictly constant	Variable (bursts allowed)
Burst handling	Bursts are absorbed or dropped	Bursts are allowed (uses saved tokens)
Token concept	No tokens — just a drain	Tokens accumulate and enable bursts

Flexibility	Less flexible — rigid output	More flexible — burst + avg control
When no data	Nothing happens	Tokens keep accumulating
Use case	Traffic policing	Traffic shaping, internet bandwidth
Example	ATM (old networks)	Modern internet, AWS bandwidth limiting

Chapter 6: Application Layer Protocols

The **Application Layer** is Layer 7 — the topmost layer of the OSI model. It is the layer that users interact with directly. It provides protocols for specific applications like web browsing, email, file transfer, and remote login.

Real-world analogy: If the Network Layer is the road system and Transport Layer is the vehicle, the Application Layer is the actual destination — the shop, hospital, or office you're trying to reach. It's where the real work happens for the user.

6.1 DNS — Domain Name System

DNS translates human-friendly domain names (like `www.google.com`) into IP addresses (like `142.250.80.4`) that computers can understand. Without DNS, you'd have to remember IP addresses to visit websites!

Real-world analogy: DNS is the internet's phone book. You look up a name (`google.com`), and it gives you the number (IP address). Just like searching a contact by name instead of remembering their 10-digit phone number.

How DNS Resolution Works (step by step):

Step	What Happens	Who Does It
1	You type <code>www.google.com</code> in browser	You (user)
2	Browser checks its own cache first	Browser / OS
3	If not cached, asks the Recursive Resolver	Your ISP's DNS server
4	Resolver asks Root DNS Server ('where is <code>.com</code> ?')	Root DNS (13 in world)
5	Root says: ask TLD Server for <code>.com</code>	Root DNS Server
6	TLD Server says: ask Google's Name Server	TLD (<code>.com</code>) Server
7	Google's Name Server gives the IP address	Authoritative DNS
8	Resolver returns IP to your browser	Recursive Resolver
9	Browser connects to <code>142.250.80.4</code>	Your computer

DNS Key Facts

- Uses Port 53 | Uses both UDP (for queries) and TCP (for zone transfers)
- DNS Record Types: A (IPv4 address), AAAA (IPv6), MX (mail server), CNAME (alias), NS (name server)
- DNS Caching: Results are cached to speed up future lookups (TTL = Time To Live)
- DNS Hierarchy: Root (`.root`) → TLD (`.com`, `.in`, `.org`) → Domain (`google.com`) → Subdomain (`www.google.com`)

6.2 HTTP & HTTPS

HTTP (HyperText Transfer Protocol) is the foundation of data communication on the web. Your browser uses HTTP to request web pages from servers. **HTTPS** is the secure version — it encrypts all data using SSL/TLS.

Real-world analogy: HTTP is like sending a postcard — anyone who handles it can read it. HTTPS is like sending a sealed, locked envelope — only the sender and receiver can read the contents. Always use HTTPS

for sensitive data!

Feature	HTTP	HTTPS
Full Name	HyperText Transfer Protocol	HTTP Secure
Port	80	443
Security	No encryption (plain text)	Encrypted with SSL/TLS
URL starts	http://	https://
Use case	Old/non-sensitive websites	Banking, shopping, login pages
Analogy	Open postcard	Sealed, locked envelope
Data safety	Anyone can read it	Only sender & receiver can read

HTTP Methods (Request Types):

- GET — Retrieve data from server (like opening a webpage)
- POST — Send data to server (like submitting a login form)
- PUT — Update existing data on server
- DELETE — Delete data on server

6.3 FTP — File Transfer Protocol

FTP is used to transfer files between a client and a server over a network. It uses TWO connections simultaneously: one for control (commands) and one for data (actual file transfer).

Real-world analogy: FTP is like a courier service. You call them (control connection on port 21) to tell them what to pick up. Then a separate van (data connection on port 20) comes to actually carry the goods.

FTP Key Facts

- Uses Port 21 for control (commands like LIST, RETR, STOR)
- Uses Port 20 for data transfer (actual file content)
- Two modes: Active FTP (server initiates data connection) and Passive FTP (client initiates both)
- Plain FTP has NO encryption — passwords sent as plain text!
- Secure alternatives: SFTP (SSH File Transfer Protocol) or FTPS (FTP + SSL)
- Common uses: Website deployment, uploading files to hosting servers

6.4 Email Protocols — SMTP, POP3, and IMAP

Sending and receiving emails involves THREE different protocols, each with a specific role.

Real-world analogy: Imagine email like a physical postal system: SMTP is the POST OFFICE that collects and delivers your letter. POP3 is like picking up your letters from your letterbox and taking them home (they leave the post office). IMAP is like reading letters at the post office itself — they stay there, accessible from anywhere.

Protocol	Full Name	Port	Role	Analogy
SMTP	Simple Mail Transfer Protocol	25/587	SENDING emails	Post office dispatch desk
POP3	Post Office Protocol v3	110	RECEIVING (downloads & deletes)	Picking letters home from mailbox
IMAP	Internet Message Access Protocol	143	RECEIVING (stays on server)	Reading mail at post office

POP3 vs IMAP — Key Difference:

- POP3: Downloads emails to your device and DELETES from server. Works offline. But if you change device, old emails are gone. Best for one device.
- IMAP: Keeps emails on the server. You see the SAME inbox from phone, laptop, tablet. Deleting on one device deletes everywhere. Best for multiple devices (modern approach — Gmail, Outlook use IMAP).

6.5 Remote Login — Telnet and SSH

Remote login protocols let you control a computer from another location over the network — like sitting at that computer's keyboard, but from far away.

Real-world analogy: Remote login is like a remote control for a computer. Telnet is an old remote control that sends commands over an open (unsecured) channel — anyone can hear what buttons you press! SSH is a modern remote control with full encryption — nobody can eavesdrop.

Feature	Telnet	SSH
Full Name	Teletype Network	Secure Shell
Port	23	22
Security	NO encryption (plain text)	Fully encrypted
Password	Sent as plain text (dangerous!)	Encrypted
Use Today	Rarely — insecure	Standard for all remote admin
Analogy	Walkie-talkie in public	Encrypted private phone line
Common Use	Legacy systems only	Linux servers, AWS, GitHub

6.6 File Sharing — NFS and SMB

File sharing protocols allow computers on a network to access and share files and folders as if they were stored locally on that computer.

Real-world analogy: NFS and SMB are like a shared Google Drive, but for local networks. Instead of storing a file on one computer and emailing it to others, everyone can directly access the same folder over the network.

Feature	NFS (Network File System)	SMB (Server Message Block)
Made By	Sun Microsystems (1984)	IBM, Microsoft
Best For	Linux/Unix environments	Windows environments
Port	2049	445 (modern), 139 (older)
Use	Sharing files across Unix/Linux servers	Windows shared folders (\\server\share)
Analogy	Shared drive in a Linux lab	Windows 'My Network Places' / shared folder
Modern Use	Cloud storage (NFS v4), Kubernetes	Office Windows file sharing, Samba

6.7 SNMP — Simple Network Management Protocol

SNMP is used to **monitor and manage network devices** — routers, switches, servers, printers — from a central location. It lets network administrators check device status, collect statistics, and configure devices remotely.

Real-world analogy: SNMP is like a hospital's nurse monitoring system. The central monitoring station (SNMP Manager) keeps checking the vital signs (CPU, memory, bandwidth) of all patients (network devices). If something goes wrong, an alarm (SNMP Trap) is sent automatically.

Three main SNMP components:

- **SNMP Manager:** The central controller — a software that collects and displays network data (e.g., Nagios, Zabbix)

- **SNMP Agent:** Software running ON each network device (router, switch) — collects local data and responds to manager queries
- **MIB (Management Information Base):** A database/dictionary that defines what data can be monitored (e.g., interface status, packet count, CPU usage)

SNMP Key Facts

- Uses Port 161 (for queries) and Port 162 (for traps/alerts)
- Uses UDP protocol (fast, no connection overhead needed for monitoring)
- SNMP Operations: GET (read a value), SET (change a value), TRAP (device alerts manager)
- Versions: SNMPv1 (basic), SNMPv2c (improved), SNMPv3 (with authentication & encryption)
- Example: Manager asks router: 'How many packets did you drop today?' Agent replies with count
- Used by: Network Operations Centers (NOC), cloud providers, enterprise IT teams

Chapter 7: Emerging Networking Concepts

Modern networking is evolving rapidly. Beyond traditional networks, new paradigms like Cloud Networking, IoT, and SDN are reshaping how we design and manage networks. These concepts are critical for placement exams and real-world industry roles.

7.1 Cloud Networking Basics

Cloud Networking refers to networking resources — routers, switches, firewalls, load balancers — that are hosted and delivered over the internet by cloud providers (AWS, Azure, Google Cloud) instead of physical hardware in your own office.

Real-world analogy: Traditional networking is like owning and maintaining your own generator for electricity. Cloud networking is like using electricity from the city grid — you don't own the infrastructure, you just pay for what you use, and it's always available.

- **Virtual Networks:** Instead of physical cables, you create networks in software (VPCs — Virtual Private Clouds)
- **Scalability:** Need more bandwidth? Just click a button. No physical hardware to buy.
- **On-Demand:** Pay only for what you use (pay-as-you-go model)
- **Global reach:** Your network can span datacenters across the world in minutes
- **Examples:** AWS VPC, Azure Virtual Network, Google Cloud VPN

Key Cloud Networking Concepts

- **VPC (Virtual Private Cloud):** Your own isolated private network inside the cloud
- **CDN (Content Delivery Network):** Servers placed near users worldwide for fast content delivery (e.g., Cloudflare, Akamai)
- **Load Balancer:** Distributes incoming traffic across multiple servers to avoid overload
- **Firewall-as-a-Service:** Security rules managed in the cloud, not on physical boxes
- **SD-WAN:** Software-Defined Wide Area Network — manage branch office networks from cloud

7.2 Internet of Things (IoT)

IoT refers to everyday physical devices — appliances, vehicles, sensors, machines — that are connected to the internet and can collect, send, and receive data without human interaction.

Real-world analogy: IoT is like giving a brain and voice to everyday objects. Your refrigerator orders milk when it runs out. Your AC turns on before you reach home. A factory machine sends an alert when it needs maintenance. Objects are now smart, connected, and talk to each other!

Category	IoT Examples	What They Do
Smart Home	Smart bulb, AC, TV, door lock	Control via phone app or voice
Healthcare	Smartwatch, glucose monitor, pacemaker	Track health, alert doctors
Agriculture	Soil sensors, drone sprayers	Monitor crop health, save water
Industry (IIoT)	Factory sensors, conveyor monitors	Predictive maintenance, efficiency
Smart City	Traffic lights, parking sensors	Reduce traffic, save energy
Transport	GPS trackers, self-driving cars	Navigation, safety, efficiency

IoT Challenges

- Security: Millions of small devices with weak security = huge attack surface
- Power: Most IoT devices run on batteries — protocols must be very power-efficient
- Scale: Billions of devices all communicating — network must handle massive scale
- Heterogeneity: Devices from different makers must all talk to each other
- Bandwidth: Devices often have very limited bandwidth — need lightweight protocols (MQTT, CoAP)

7.3 MQTT — Message Queuing Telemetry Transport

MQTT is a lightweight messaging protocol designed specifically for IoT devices that have limited processing power, battery, and bandwidth. It uses a **publish-subscribe** model rather than the traditional request-response model.

Real-world analogy: MQTT is like a newspaper subscription service. You (subscriber) tell the newspaper agency (broker): 'Send me the Sports section whenever it's available.' The journalist (publisher) just writes and submits articles — they don't know or care who reads them. The agency (broker) manages everything in between.

Publish-Subscribe Model:

- **Publisher:** A device that sends (publishes) data to a topic. Example: Temperature sensor publishes to topic 'home/living-room/temperature'
- **Broker:** A central server that receives all messages and distributes them. Example: Mosquitto broker, AWS IoT Core
- **Subscriber:** A device or app that receives messages from a topic. Example: Your phone app subscribes to 'home/living-room/temperature' and shows the value

MQTT Key Facts

- Full Name: Message Queuing Telemetry Transport
- Port: 1883 (plain) | 8883 (with TLS encryption)
- Transport: Runs over TCP/IP
- QoS Levels: 0 (at most once), 1 (at least once), 2 (exactly once)
- Very lightweight: Tiny header (2 bytes minimum) — perfect for constrained devices
- Used by: Facebook Messenger (originally!), smart home systems, industrial IoT, AWS IoT

7.4 CoAP — Constrained Application Protocol

CoAP is another IoT protocol, but it follows the same **request-response** model as HTTP — making it easy for web developers to use. However, it is much lighter than HTTP and works over UDP instead of TCP.

Real-world analogy: CoAP is like a simplified, lightweight version of HTTP designed for tiny devices. If HTTP is a full-sized bus for carrying web traffic, CoAP is a bicycle — same roads (internet), much smaller and faster for short trips.

Feature	CoAP	MQTT	HTTP
Model	Request-Response (like HTTP)	Publish-Subscribe	Request-Response
Transport	UDP (fast, lightweight)	TCP (reliable)	TCP
Overhead	Very low	Very low	High
Best for	Direct device-to-device comm	Many devices to one broker	Web browsers
Reliability	Optional (CON messages)	QoS levels 0,1,2	TCP guarantees
Port	5683	1883	80/443
Use case	Smart meters, sensors	Smart home, industrial IoT	Web apps

7.5 Software-Defined Networking (SDN)

SDN is a revolutionary approach to networking where the **control plane** (the brain that decides where traffic goes) is separated from the **data plane** (the hardware that actually forwards traffic). The entire network is controlled by software from a central controller.

Real-world analogy: Traditional networking is like having a separate driver in every vehicle — each car (router/switch) makes its own decisions. SDN is like having one central traffic control tower that tells every vehicle exactly where to go at every moment. The vehicles just follow instructions — they don't think.

Traditional Network vs SDN:

- Traditional: Each router/switch has its own control logic (brain inside the box). Hard to change, expensive hardware, slow to reconfigure.
- SDN: A central SOFTWARE controller manages all network devices. Devices become simple forwarding machines. Control is programmable and flexible.

Three Layers of SDN Architecture:

Layer	What It Contains	Example
Application Layer	Business logic, network apps	Load balancer app, firewall app, monitoring tool
Control Layer	SDN Controller (the BRAIN)	OpenDaylight, ONOS, Cisco ACI, VMware NSX
Data/Infrastructure Layer	Physical network devices	Switches, routers — just forward packets as told

SDN Key Benefits

- Centralized control: One controller manages entire network — simpler management
- Programmability: Network behavior can be changed with software, not hardware replacement
- Cost reduction: Use cheap commodity hardware instead of expensive proprietary equipment
- Agility: Respond to network changes in seconds (software update) instead of weeks (hardware)
- Used by: Google (B4 WAN), Facebook, Microsoft Azure, AWS, all major cloud providers
- Key protocol: OpenFlow — standardized communication between controller and switches

Quick Revision Summary — All Three Chapters

Real-world analogy: Read this page 15 minutes before your exam. All key facts in one place!

Chapter 5: Transport Layer — Key Points

- Transport Layer = Layer 4 = Process-to-process delivery using PORT NUMBERS
- TCP = Reliable, ordered, connection-oriented. Uses 3-way handshake (SYN, SYN-ACK, ACK)
- TCP close = 4-way handshake (FIN, ACK, FIN, ACK)
- UDP = Fast, unreliable, connectionless. No handshake. Used in video, gaming, DNS
- Port ranges: 0-1023=Well-known | 1024-49151=Registered | 49152-65535=Dynamic
- Key ports: HTTP=80, HTTPS=443, FTP=21, SSH=22, DNS=53, SMTP=25, POP3=110, IMAP=143
- Leaky Bucket: Fixed output rate, bursts dropped if bucket full — rigid traffic shaping
- Token Bucket: Tokens accumulate, bursts ALLOWED up to bucket size — flexible

Chapter 6: Application Layer — Key Points

- DNS: Translates domain names to IP. Port 53. Root→TLD→Authoritative name server
- HTTP: Web protocol, Port 80, plain text. HTTPS: Port 443, encrypted with SSL/TLS
- FTP: File transfer. Port 21 (control) + Port 20 (data). Insecure — use SFTP instead
- SMTP (Port 25/587): SENDING emails | POP3 (Port 110): Receive & DOWNLOAD | IMAP (Port 143): Receive, stays on server
- Telnet (Port 23): Insecure remote login — avoid! SSH (Port 22): Secure encrypted remote login
- NFS: File sharing for Linux/Unix networks | SMB: File sharing for Windows networks
- SNMP (Port 161/162): Monitor/manage network devices. Manager + Agent + MIB

Chapter 7: Emerging Concepts — Key Points

- Cloud Networking: Network resources (routers, firewalls) hosted in cloud. VPC, CDN, Load Balancer
- IoT: Everyday devices (fridges, sensors, cars) connected to internet. Challenges: security, power, scale
- MQTT: Publish-Subscribe protocol for IoT. Broker-based. Port 1883. Runs on TCP. Very lightweight
- CoAP: HTTP-like protocol for IoT. Runs on UDP. Port 5683. Request-Response model. Very lightweight
- SDN: Separates control plane (brain) from data plane (forwarding). Central software controller
- SDN layers: Application → Control (controller) → Data/Infrastructure (switches/routers)
- SDN protocol: OpenFlow. Used by: Google, AWS, Azure, Facebook in their data centers

Memory Tricks for Exam

Topic	Easy Memory Trick
3-Way Handshake	SYN → SYN-ACK → ACK = 'Hello?' → 'Hello! Can you hear me?' → 'Yes!'
4-Way Handshake	FIN, ACK, FIN, ACK = 'I'm done' → 'OK' → 'Me too' → 'OK bye!'
TCP vs UDP	TCP = Registered post (slow, guaranteed). UDP = Postcard (fast, might lose)
Leaky vs Token Bucket	Leaky = Always drips same rate (rigid). Token = Save tokens, burst later (flexible)
SMTP/POP3/IMAP	SMTP = Sends. POP3 = Pulls & Purges. IMAP = Inbox stays on server
Telnet vs SSH	Telnet = Open walkie-talkie. SSH = Encrypted private call

MQTT vs CoAP	MQTT = Pub-Sub (via broker). CoAP = Request-Response (like HTTP but tiny)
SDN	Traditional = Each router is its own boss. SDN = One boss controls all routers